

一样。

用于初始化 `vector` 对象的值也可能仅是数组的一部分：

```
// 拷贝三个元素: int_arr[1]、int_arr[2]、int_arr[3]
vector<int> subVec(int_arr + 1, int_arr + 4);
```

这条初始化语句用 3 个元素创建了对象 `subVec`，3 个元素的值分别来自 `int_arr[1]`、`int_arr[2]` 和 `int_arr[3]`。

建议：尽量使用标准库类型而非数组

使用指针和数组很容易出错。一部分原因是概念上的问题：指针常用于底层操作，因此容易引发一些与烦琐细节有关的错误。其他问题则源于语法错误，特别是声明指针时的语法错误。

现代的 C++ 程序应当尽量使用 `vector` 和迭代器，避免使用内置数组和指针；应该尽量使用 `string`，避免使用 C 风格的基于数组的字符串。

3.5.5 节练习

练习 3.41：编写一段程序，用整型数组初始化一个 `vector` 对象。

练习 3.42：编写一段程序，将含有整数元素的 `vector` 对象拷贝给一个整型数组。



3.6 多维数组

严格来说，C++ 语言中没有多维数组，通常所说的多维数组其实是数组的数组。谨记这一点，对今后理解和使用多维数组大有益处。

当一个数组的元素仍然是数组时，通常使用两个维度来定义它：一个维度表示数组本身大小，另外一个维度表示其元素（也是数组）大小：

```
int ia[3][4]; // 大小为 3 的数组，每个元素是含有 4 个整数的数组
// 大小为 10 的数组，它的每个元素都是大小为 20 的数组，
// 这些数组的元素是含有 30 个整数的数组
int arr[10][20][30] = {0}; // 将所有元素初始化为 0
```

如 3.5.1 节（第 103 页）所介绍的，按照由内而外的顺序阅读此类定义有助于更好地理解其真实含义。在第一条语句中，我们定义的名字是 `ia`，显然 `ia` 是一个含有 3 个元素的数组。接着观察右边发现，`ia` 的元素也有自己的维度，所以 `ia` 的元素本身又都是含有 4 个元素的数组。再观察左边知道，真正存储的元素是整数。因此最后可以明确第一条语句的含义：它定义了一个大小为 3 的数组，该数组的每个元素都是含有 4 个整数的数组。

使用同样的方式理解 `arr` 的定义。首先 `arr` 是一个大小为 10 的数组，它的每个元素都是大小为 20 的数组，这些数组的元素又都是含有 30 个整数的数组。实际上，定义数组时对下标运算符的数量并没有限制，因此只要愿意就可以定义这样一个数组：它的元素还是数组，下一级数组的元素还是数组，再下一级数组的元素还是数组，以此类推。

对于二维数组来说，常把第一个维度称作行，第二个维度称作列。

多维数组的初始化

允许使用花括号括起来的一组值初始化多维数组，这点和普通的数组一样。下面的初始化形式中，多维数组的每一行分别用花括号括了起来：

```
int ia[3][4] = {           // 三个元素，每个元素都是大小为 4 的数组
    {0, 1, 2, 3},         // 第 1 行的初始值
    {4, 5, 6, 7},         // 第 2 行的初始值
    {8, 9, 10, 11}        // 第 3 行的初始值
};
```

其中内层嵌套着的花括号并非必需的，例如下面的初始化语句，形式上更为简洁，完成的功能和上面这段代码完全一样：

```
// 没有标识每行的花括号，与之前的初始化语句是等价的
int ia[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

类似于一维数组，在初始化多维数组时也并非所有元素的值都必须包含在初始化列表之内。如果仅仅想初始化每一行的第一个元素，通过如下的语句即可：

```
// 显式地初始化每行的首元素
int ia[3][4] = {{ 0 }, { 4 }, { 8 }};
```

127

其他未列出的元素执行默认值初始化，这个过程和一维数组（参见 3.5.1 节，第 102 页）一样。在这种情况下如果再省略掉内层的花括号，结果就大不一样了。下面的代码

```
// 显式地初始化第 1 行，其他元素执行值初始化
int ix[3][4] = {0, 3, 6, 9};
```

含义发生了变化，它初始化的是第一行的 4 个元素，其他元素被初始化为 0。

多维数组的下标引用

可以使用下标运算符来访问多维数组的元素，此时数组的每个维度对应一个下标运算符。

如果表达式含有的下标运算符数量和数组的维度一样多，该表达式的结果将是给定类型的元素；反之，如果表达式含有的下标运算符数量比数组的维度小，则表达式的结果将是给定索引处的一个内层数组：

```
// 用 arr 的首元素为 ia 最后一行的最后一个元素赋值
ia[2][3] = arr[0][0][0];
int (&row)[4] = ia[1]; // 把 row 绑定到 ia 的第二个 4 元素数组上
```

在第一个例子中，对于用到的两个数组来说，表达式提供的下标运算符数量都和它们各自的维度相同。在等号左侧，`ia[2]` 得到数组 `ia` 的最后一行，此时返回的是表示 `ia` 最后一行的那个一维数组而非任何实际元素；对这个一维数组再取下标，得到编号为 `[3]` 的元素，也就是这一行的最后一个元素。

类似的，等号右侧的运算对象包含 3 个维度。首先通过索引 0 得到最外层的数组，它是一个大小为 20 的（多维）数组；接着获取这 20 个元素数组的第一个元素，得到一个大小为 30 的一维数组；最后再取出其中的第一个元素。

在第二个例子中，把 `row` 定义成一个含有 4 个整数的数组的引用，然后将其绑定到 `ia` 的第 2 行。

再举一个例子，程序中经常会用到两层嵌套的 `for` 循环来处理多维数组的元素：

```
constexpr size_t rowCnt = 3, colCnt = 4;
```

```
int ia[rowCnt][colCnt]; // 12 个未初始化的元素
// 对于每一行
for (size_t i = 0; i != rowCnt; ++i) {
    // 对于行内的每一列
    for (size_t j = 0; j != colCnt; ++j) {
        // 将元素的位置索引作为它的值
        ia[i][j] = i * colCnt + j;
    }
}
```

外层的 for 循环遍历 ia 的所有元素，注意这里的元素是一维数组；内层的 for 循环则遍历那些一维数组的整数元素。此例中，我们将元素的值设为该元素在整个数组中的序号。



使用范围 for 语句处理多维数组

128

由于在 C++11 新标准中新增了范围 for 语句，所以前一个程序可以简化为如下形式：

```
size_t cnt = 0;
for (auto &row : ia)           // 对于外层数组的每一个元素
    for (auto &col : row) {     // 对于内层数组的每一个元素
        col = cnt;             // 将下一个值赋给该元素
        ++cnt;                 // 将 cnt 加 1
    }
```

这个循环赋给 ia 元素的值和之前那个循环是完全相同的，区别之处是通过使用范围 for 语句把管理数组索引的任务交给了系统来完成。因为要改变元素的值，所以得把控制变量 row 和 col 声明成引用类型（参见 3.2.3 节，第 83 页）。第一个 for 循环遍历 ia 的所有元素，这些元素是大小为 4 的数组，因此 row 的类型就应该是含有 4 个整数的数组的引用。第二个 for 循环遍历那些 4 元素数组中的某一个，因此 col 的类型是整数的引用。每次迭代把 cnt 的值赋给 ia 的当前元素，然后将 cnt 加 1。

在上面的例子中，因为要改变数组元素的值，所以我们选用引用类型作为循环控制变量，但其实还有一个深层次的原因促使我们这么做。举一个例子，考虑如下的循环：

```
for (const auto &row : ia)     // 对于外层数组的每一个元素
    for (auto col : row)       // 对于内层数组的每一个元素
        cout << col << endl;
```

这个循环中并没有任何写操作，可是我们还是将外层循环的控制变量声明成了引用类型，这是为了避免数组被自动转成指针（参见 3.5.3 节，第 105 页）。假设不用引用类型，则循环如下述形式：

```
for (auto row : ia)
    for (auto col : row)
```

程序将无法通过编译。这是因为，像之前一样第一个循环遍历 ia 的所有元素，注意这些元素实际上是大小为 4 的数组。因为 row 不是引用类型，所以编译器初始化 row 时会自动将这些数组形式的元素（和其他类型的数组一样）转换成指向该数组内首元素的指针。这样得到的 row 的类型就是 int*，显然内层的循环就不合法了，编译器将试图在一个 int* 内遍历，这显然和程序的初衷相去甚远。



要使用范围 for 语句处理多维数组，除了最内层的循环外，其他所有循环的控制变量都应该是引用类型。

指针和多维数组

当程序使用多维数组的名字时，也会自动将其转换成指向数组首元素的指针。

Note

定义指向多维数组的指针时，千万别忘了这个多维数组实际上是数组的数组。

129

因为多维数组实际上是数组的数组，所以由多维数组名转换得来的指针实际上是指向第一个内层数组的指针：

```
int ia[3][4];           // 大小为 3 的数组，每个元素是含有 4 个整数的数组
int (*p)[4] = ia;       // p 指向含有 4 个整数的数组
p = &ia[2];             // p 指向 ia 的尾元素
```

根据 3.5.1 节（第 103 页）提出的策略，我们首先明确 (*p) 意味着 p 是一个指针。接着观察右边发现，指针 p 所指的是一个维度为 4 的数组；再观察左边知道，数组中的元素是整数。因此，p 就是指向含有 4 个整数的数组的指针。

Note

在上述声明中，圆括号必不可少：

```
int *ip[4];             // 整型指针的数组
int (*ip)[4];           // 指向含有 4 个整数的数组
```

随着 C++11 新标准的提出，通过使用 auto 或者 decltype（参见 2.5.2 节，第 61 页）就能尽可能地避免在数组前面加上一个指针类型了：

C++
11

```
// 输出 ia 中每个元素的值，每个内层数组各占一行
// p 指向含有 4 个整数的数组
for (auto p = ia; p != ia + 3; ++p) {
    // q 指向 4 个整数数组的首元素，也就是说，q 指向一个整数
    for (auto q = *p; q != *p + 4; ++q)
        cout << *q << ' ';
    cout << endl;
}
```

外层的 for 循环首先声明一个指针 p 并令其指向 ia 的第一个内层数组，然后依次迭代直到 ia 的全部 3 行都处理完为止。其中递增运算 ++p 负责将指针 p 移动到 ia 的下一行。

内层的 for 循环负责输出内层数组所包含的值。它首先令指针 q 指向 p 当前所在行的第一个元素。*p 是一个含有 4 个整数的数组，像往常一样，数组名被自动地转换成指向该数组首元素的指针。内层 for 循环不断迭代直到我们处理完了当前内层数组的所有元素为止。为了获取内层 for 循环的终止条件，再一次解引用 p 得到指向内层数组首元素的指针，给它加上 4 就得到了终止条件。

当然，使用标准库函数 begin 和 end（参见 3.5.3 节，第 106 页）也能实现同样的功能，而且看起来更简洁一些：

```
// p 指向 ia 的第一个数组
for (auto p = begin(ia); p != end(ia); ++p) {
    // q 指向内层数组的首元素
    for (auto q = begin(*p); q != end(*p); ++q)
        cout << *q << ' '; // 输出 q 所指的整数值
    cout << endl;
}
```

130 在这一版本的程序中，循环终止条件由 `end` 函数负责判断。虽然我们也能推断出 `p` 的类型是指向含有 4 个整数的数组的指针，`q` 的类型是指向整数的指针，但是使用 `auto` 关键字我们就不必再烦心这些类型到底是什么了。

类型别名简化多维数组的指针

读、写和理解一个指向多维数组的指针是一个让人不胜其烦的工作，使用类型别名（参见 2.5.1 节，第 60 页）能让这项工作变得简单一点儿，例如：

```
using int_array = int[4]; // 新标准下类型别名的声明，参见 2.5.1 节（第 60 页）
typedef int int_array[4]; // 等价的 typedef 声明，参见 2.5.1 节（第 60 页）

// 输出 ia 中每个元素的值，每个内层数组各占一行
for (int_array *p = ia; p != ia + 3; ++p) {
    for (int *q = *p; q != *p + 4; ++q)
        cout << *q << ' ';
    cout << endl;
}
```

程序将类型“4 个整数组成的数组”命名为 `int_array`，用类型名 `int_array` 定义外层循环的控制变量让程序显得简洁明了。

3.6 节练习

练习 3.43：编写 3 个不同版本的程序，令其均能输出 `ia` 的元素。版本 1 使用范围 `for` 语句管理迭代过程；版本 2 和版本 3 都使用普通的 `for` 语句，其中版本 2 要求用下标运算符，版本 3 要求用指针。此外，在所有 3 个版本的程序中都要直接写出数据类型，而不能使用类型别名、`auto` 关键字或 `decltype` 关键字。

练习 3.44：改写上一个练习中的程序，使用类型别名来代替循环控制变量的类型。

练习 3.45：再一次改写程序，这次使用 `auto` 关键字。

小结

131

`string` 和 `vector` 是两种最重要的标准库类型。`string` 对象是一个可变长的字符序列，`vector` 对象是一组同类型对象的容器。

迭代器允许对容器中的对象进行间接访问，对于 `string` 对象和 `vector` 对象来说，可以通过迭代器访问元素或者在元素间移动。

数组和指向数组元素的指针在一个较低的层次上实现了与标准库类型 `string` 和 `vector` 类似的功能。一般来说，应该优先选用标准库提供的类型，之后再考虑 C++ 语言内置的低层的替代品数组或指针。

术语表

begin 是 `string` 和 `vector` 的成员，返回指向第一个元素的迭代器。也是一个标准库函数，输入一个数组，返回指向该数组首元素的指针。

缓冲区溢出 (buffer overflow) 一种严重的程序故障，主要的原因是试图通过一个越界的索引访问容器内容，容器类型包括 `string`、`vector` 和数组等。

C 风格字符串 (C-style string) 以空字符结束的字符数组。字符串字面值是 C 风格字符串，C 风格字符串容易出错。

类模板 (class template) 用于创建具体类类型的模板。要想使用类模板，必须提供关于类型的辅助信息。例如，要定义一个 `vector` 对象需要指定元素的类型：`vector<int>` 包含 `int` 类型的元素。

编译器扩展 (compiler extension) 某个特定的编译器为 C++ 语言额外增加的特性。基于编译器扩展编写的程序不易移植到其他编译器上。

容器 (container) 是一种类型，其对象容纳了一组给定类型的对象。`vector` 是一种容器类型。

拷贝初始化 (copy initialization) 使用赋值号 (=) 的初始化形式。新创建的对象是初始值的一个副本。

difference_type 由 `string` 和 `vector` 定义的一种带符号整数类型，表示两个迭代

器之间的距离。

直接初始化 (direct initialization) 不使用赋值号 (=) 的初始化形式。

empty 是 `string` 和 `vector` 的成员，返回一个布尔值。当对象的大小为 0 时返回真，否则返回假。

end 是 `string` 和 `vector` 的成员，返回一个尾后迭代器。也是一个标准库函数，输入一个数组，返回指向该数组尾元素的下一位置的指针。

getline 在 `string` 头文件中定义的一个函数，以一个 `istream` 对象和一个 `string` 对象为输入参数。该函数首先读取输入流的内容直到遇到换行符停止，然后将读入的数据存入 `string` 对象，最后返回 `istream` 对象。其中换行符读入但是不保留。

索引 (index) 是下标运算符使用的值。表示要在 `string` 对象、`vector` 对象或者数组中访问的一个位置。

实例化 (instantiation) 编译器生成一个指定的模板类或函数的过程。

迭代器 (iterator) 是一种类型，用于访问容器中的元素或者在元素之间移动。

迭代器运算 (iterator arithmetic) 是 `string` 或 `vector` 的迭代器的运算：迭代器与整数相加或相减得到一个新的迭代器，与原来的迭代器相比，新迭代器向前